Predicting Chess Game Outcomes from Pre-Game and Early-Game Features: A Comparison of Machine Learning Methods

CS-412 Intermediate Project Report

Hashim Khan
University of Illinois Chicago
Chicago, Illinois, USA

Abstract

This report describes progress on an application project that predicts the outcome of online chess games from player ratings, time control, and opening. Using 19,113 Lichess games from Kaggle, we built a leakage-free preprocessing pipeline, defined a "higher-rated player wins" baseline, and trained five standard classifiers on a binary (White vs. Black) and a three-class (with draws) task. We summarize the data, the work completed so far, preliminary results, and the remaining plan for the final report.

1 Introduction and Problem Description

Chess is a two-player game with a large public record of games, and every game ends in a White win, a Black win, or a draw. Online platforms such as Lichess rate each player with a numerical rating that estimates playing strength, in the tradition of the Elo [4] and Glicko [5] systems, which model win probability as a function of the rating difference. This project asks: *how well can machine learning models predict the outcome of an online chess game from information available before or early in the game, and does a learned model improve on the simple rule that the higher-rated player wins?*

We study two prediction tasks on the same data. The **binary task** predicts whether White or Black wins and excludes draws. The **three-class task** also predicts draws, which are rare, so it additionally tests how models cope with class imbalance. We address three questions:

(1) Do machine learning models beat the baseline of "the higher-rated player wins"?
(2) Which features matter most: ratings, opening, or time control?
(3) How does accuracy change as more of the game is revealed (e.g., the first 5 or 10 moves)?

The first question has a preliminary answer in this report. Questions 2 and 3 require the feature-importance and early-move experiments described in Section 5.

A central methodological point is **data leakage**. Several columns in the raw data, such as how the game ended or how many moves it lasted, are only known after the game is over. A model using them would look accurate without being useful for prediction, so we exclude them (Section 3.3).

2 Related Work

Rating systems [4, 5] give a principled prediction from rating difference alone, which motivates our baseline. For tabular data like ours, tree ensembles such as Random Forests [1] and gradient boosting [2] are standard strong methods, and a recent large benchmark found that tree-based models still outperform neural networks on medium-sized tabular datasets of about 10,000 samples [6]. This is why we focus on classical models here. Deep learning has been applied to chess directly, as in DeepChess [3] and AlphaZero [9], but those methods work on board positions and require far more data and compute. Our work instead compares standard classifiers on summary features, with explicit leakage control and a study of how much information early moves add.

3 Data

3.1 Source and size

We use the Lichess games dataset on Kaggle [7], which contains one row per game with player ratings, the time control, the opening, the full move list, and the result. The file has 20,058 rows. After removing duplicate game IDs and any rows with missing values or an invalid result (945 rows removed), 19,113 games remain. Ratings range from 784 to 2723 with a mean of 1594. The games use 365 distinct ECO opening codes.

3.2 Target distribution

Of the 19,113 games, 49.9% are White wins, 45.4% are Black wins, and 4.6% (888 games) are draws. The binary task therefore uses 18,225 games, and draws are a small minority class in the three-class task.

3.3 Features and leakage control

We use the following features, all available before the game starts or determined by its opening moves:

- `white_rating`, `black_rating`, their difference, and their mean (we treat the recorded ratings as the players' ratings at the start of the game);
- time control: initial time in minutes and increment in seconds, parsed from `increment_code`;
- whether the game was rated;
- opening: the ECO letter (A–E), the 30 most frequent ECO codes (others grouped as "other"), and `opening_ply`.

We exclude `victory_status` (how the game ended), `turns` (total number of moves), `created_at` and `last_move_at` (their difference is the game duration), the full `moves` list (we will only use its first N moves later), and the player IDs, which would let a model memorize individual players.

Table 1 summarizes how each raw column was handled.

Table 1: Handling of the raw columns.

Column(s)	Decision	Reason
`white_rating`, `black_rating`	Use	Known before the game
`increment_code`	Use	Time control is fixed in advance
`rated`	Use	Known before the game
`opening_eco`, `opening_ply`	Use	Set by the first moves
`victory_status`	Exclude	Reveals how the game ended
`turns`	Exclude	Game length is known only at the end
`created_at`, `last_move_at`	Exclude	Difference gives game duration
`moves`	Later	Only the first N moves will be used
`id`, player IDs	Exclude	Identifiers; allow memorization

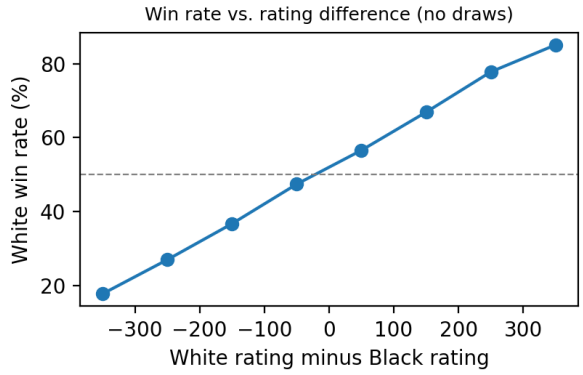


Figure 1: White's win rate by rating difference (White minus Black), in bins of 100 rating points, draws excluded.

3.4 Exploratory analysis

Figure 1 shows White's win rate (draws excluded) as a function of the rating difference. White's win rate rises steadily from 17.7% when Black is rated 300–400 points higher to 85.1% when White is rated 300–400 points higher, so rating difference is a strong signal and the rating-based rule is a natural baseline.

4 Work Completed So Far

4.1 Pipeline

We implemented the full pipeline in a single Python script (`chess_baseline.py`, in our GitHub repository) using pandas and scikit-learn. It loads and cleans the data, builds the features above, and splits the data into 80% training and 20% test sets, stratified by outcome with a fixed random seed. For the binary task this gives 14,580 training and 3,645 test games. Numeric features are standardized and categorical features are one-hot encoded inside each model pipeline, so no information from the test set is used in preprocessing.

4.2 Baselines and models

We evaluate two baselines: always predicting the majority class, and predicting that the higher-rated player wins (White if the ratings are equal). We then train five classifiers with scikit-learn [8]: Logistic Regression, a depth-5 Decision Tree, k-Nearest Neighbors ($k = 25$), a Random Forest with 200 trees, and Gradient Boosting. These

Table 2: Binary task (White vs. Black, draws excluded). Accuracy and macro-F1 in %.

Model	CV acc.	Test acc.	Test F1
Majority class	–	52.4	34.4
Higher-rated player wins	–	65.2	65.2
Logistic Regression	65.0 ± 0.8	65.0	64.7
Decision Tree (depth 5)	64.8 ± 0.8	64.1	63.2
k-NN (k=25)	62.9 ± 0.7	63.3	63.0
Random Forest (200 trees)	63.1 ± 0.8	63.2	63.1
Gradient Boosting	65.1 ± 0.7	65.5	65.1

Table 3: Three-class task (White, Black, draw). Accuracy and macro-F1 in %.

Model	CV acc.	Test acc.	Test F1
Majority class	–	49.9	22.2
Higher-rated player wins	–	62.3	42.5
Logistic Regression	61.9 ± 0.8	62.4	42.4
Decision Tree (depth 5)	61.7 ± 0.6	62.0	41.8
k-NN (k=25)	59.0 ± 1.4	60.0	40.9
Random Forest (200 trees)	60.3 ± 0.6	60.3	42.1
Gradient Boosting	61.4 ± 0.5	62.4	43.5

use mostly default settings; hyperparameter tuning is future work. Each model is evaluated by 5-fold cross-validation on the training set (accuracy, mean ± standard deviation) and then once on the held-out test set (accuracy and macro-averaged F1).

The models cover a range of complexity. **Logistic Regression** is a linear model whose predicted probability depends on a weighted sum of the features, and is a natural fit when the signal is mostly the rating difference. The **Decision Tree** splits the data with a sequence of threshold rules and is easy to interpret, but we limit its depth to 5 to avoid overfitting. **k-Nearest Neighbors** predicts from the 25 most similar training games, and is sensitive to how features are scaled, which is why numeric features are standardized. The **Random Forest** [1] averages many decorrelated trees, and **Gradient Boosting** builds trees one after another, each correcting the errors of the previous ones.

4.3 Evaluation protocol

We report accuracy, the fraction of games predicted correctly, because the binary task is nearly balanced. For the three-class task accuracy alone is misleading: since only 4.6% of games are draws, a model that never predicts a draw can still score well. We therefore also report *macro-averaged F1*, which computes the F1 score of each class separately and averages them, so that the rare draw class counts as much as the two common classes. Cross-validation is stratified and run only on the training set; the test set is used once for the final numbers, so the reported test accuracy is not tuned to the test games.

4.4 Preliminary results

Table 2 reports the binary task and Table 3 the three-class task. Figure 2 compares test accuracy on the binary task.

None of the five models meaningfully improves on the rating-only rule. On the binary task the baseline reaches 65.2% test accuracy, and the best models, Gradient Boosting (65.5%) and Logistic

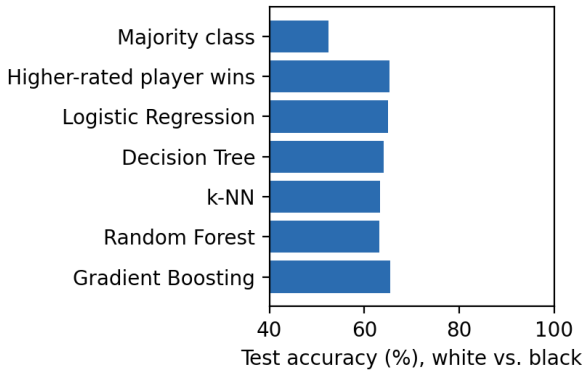


Figure 2: Test accuracy on the binary task for the baselines and the five models.

Regression (65.0%), are within a few tenths of a point of it, which is smaller than the cross-validation standard deviation of about 0.7–0.8 points. k-NN and the Random Forest are about two points lower (63.3% and 63.2%); a plausible reason is that the one-hot opening features, with hundreds of distinct opening codes, let these models fit noise, but we have not tested this yet. The three-class results are similar (baseline 62.3%, Gradient Boosting 62.4%). Macro-F1 is far below accuracy for every method (about 41–44%) and close to the baseline's, which can never predict a draw, which suggests that the models also rarely identify draws; we will check per-class results. Overall, rating difference carries most of the usable signal, and the opening and time control appear to add little with default model settings. Whether tuning or early-move features change this is the focus of the remaining work.

4.5 Challenges and lessons so far

Three issues shaped the design. First, **leakage**: several raw columns would have made the models look far better than they are, so we audited every column before modeling (Table 1). Second, **class imbalance**: draws make up only 4.6% of games, so plain accuracy is misleading in the three-class task, and we also report macro-F1. Third, **a strong baseline**: because the rating-only rule is already hard to beat, small differences between models are easy to over-interpret. We therefore compare differences against the cross-validation standard deviation.

5 Remaining Work

- **Hyperparameter tuning** (Nov 2–12): cross-validated search over tree depth, number of trees, learning rate, and k, so that model comparisons are fair.
- **Early-move features** (Nov 13–20): extract the first N moves (e.g., $N = 5, 10$) from the move list and measure how much accuracy improves, which answers research question 3.
- **Feature importance and ablations** (Nov 13–20): measure which features matter, including a model without ratings to isolate the effect of the opening and time control.
- **Handling draws** (Nov 2–12): class weighting and per-class precision and recall for the three-class task.
- **Larger sample** (optional): a sample from the Lichess open database to check that conclusions hold at larger scale.
- **Presentation and final report**: presentation from mid-November; final 8–10 page report by Dec 1 (written Nov 21–30), with a clean repository.

Table 4 summarizes the status of each project component against the milestones in the proposal.

Table 4: Project status against the proposal.

Component	Status
Data download and cleaning	Done
Leakage audit	Done
Rating baseline and majority baseline	Done
Five models, default settings	Done
Binary and three-class evaluation	Done
Hyperparameter tuning	Not started
Feature importance and ablations	Not started
Early-move features	Not started
Final report and presentation	Not started

5.1 Risks

The main risk is that the rating difference may remain by far the strongest feature, so tuning and extra features could add only small gains. This would not invalidate the project: showing carefully, with leakage control and a fair comparison, that a simple rule is hard to beat is itself a useful finding. If tuning gives no gain, the early-move experiment becomes the main contribution, since the first moves are the main information not yet used beyond ratings, opening, and time control. A second risk is dataset size: differences of a fraction of a point are within noise at about 18,225 games.

6 Code and Team

This is a solo project by Hashim Khan. All code, the proposal, and this report are maintained in the GitHub repository at https://github.com/Hashimak03/Predicting-Chess-Game-Outcomes-ML-Project-.

References

[1] Leo Breiman. 2001. Random Forests. *Machine Learning* 45, 1 (2001), 5–32.
[2] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 785–794.
[3] Omid E. David, Nathan S. Netanyahu, and Lior Wolf. 2016. DeepChess: End-to-End Deep Neural Network for Automatic Learning in Chess. In *International Conference on Artificial Neural Networks*. Springer, 88–96.
[4] Arpad E. Elo. 1978. *The Rating of Chessplayers, Past and Present.* Arco Publishing.
[5] Mark E. Glickman. 1999. Parameter Estimation in Large Dynamic Paired Comparison Experiments. *Journal of the Royal Statistical Society: Series C (Applied Statistics)* 48, 3 (1999), 377–394.
[6] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. 2022. Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.
[7] Kaggle. [n. d.]. Chess Game Dataset (Lichess). https://www.kaggle.com/datasets/datasnaek/chess. Data from Lichess.org.
[8] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, et al. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
[9] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, et al. 2018. A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play. *Science* 362, 6419 (2018), 1140–1144.